Como o Morphe funciona

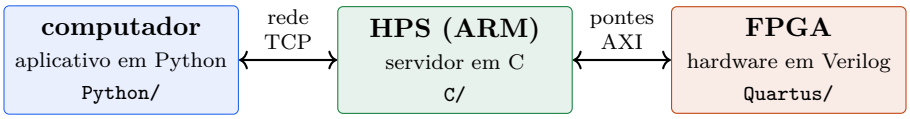
A lógica e a infraestrutura da versão ADC e FFT,
para quem vai acrescentar um módulo ou uma função

Antonio Nicassio Santos Lima Estágio LabPS/UEFS — supervisão: Prof. Armando S. Sanca

outubro de 2026

O Morphe é uma plataforma em que um sinal sai do computador, é processado em **hardware** na FPGA de uma placa DE1-SoC e volta ao computador, onde é comparado com o mesmo cálculo feito em software. Esta versão (branch `estagio/adc-fft`) tem dois blocos de hardware: a **captura pelo ADC** da placa e a **FFT de 1024 pontos**.

Este documento explica **como as peças se encaixam**. Ele não é um passo a passo: é o mapa que você precisa ter na cabeça antes de mexer. Quando o professor pedir um módulo novo (um filtro, uma operação, um medidor), você vai mexer nas mesmas camadas em que o ADC e a FFT estão, e na mesma ordem. A última parte (9) junta essa lista. O exemplo completo, feito do começo ao fim, é o **roteiro do módulo de soma** (`docs/roteiro-modulo/`).



As cores valem para o documento inteiro: **azul** é o computador, **verde** é o processador da placa (HPS) e **laranja** é a FPGA.

Sumário

1 Visão geral: o caminho de um dado **2**

2 A placa por dentro: HPS, FPGA e as pontes **3**

2.1 As duas pontes . . . 3

2.2 Memória de duas portas e PIO: as duas peças de toda interface . . . 3

2.3 O mapa de endereços . . . 3

2.4 Como o servidor sabe que a FPGA é a certa . . . 4

3 A FPGA (pasta `Quartus/`) **5**

3.1 O topo: `ghrd_top.v` . . . 5

3.2 O combinado entre o HPS e um bloco: start e done . . . 5

3.3 O ADC: `adc_captura.v` . . . 6

3.4 A FFT: `fft_wrapper.v` e o IP `fft_core` . . . 6

3.5 O sistema do Platform Designer: `soc_system.qsys` . . . 7

3.6 O que é gerado, e por quem . . . 7

4 O servidor (pasta `C/`) 8

5 O protocolo (`morphe_protocol.h` e `.py`) 9

6 O aplicativo (pasta `Python/`) 10

7 A infraestrutura: como tudo chega à placa e aos computadores 10

8 Testes que já existem 11

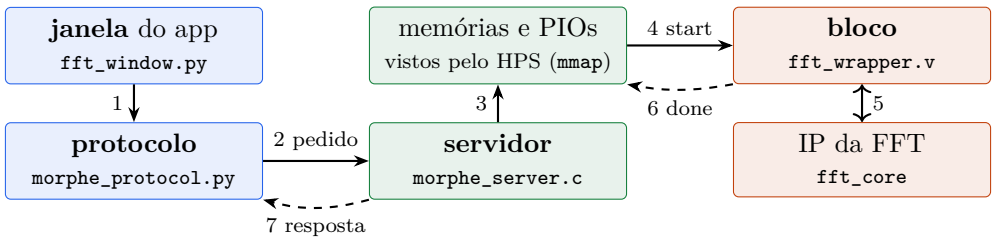
9 Acrescentar um módulo: o que mexer, camada por camada 11

Glossário 13

1 Visão geral: o caminho de um dado

A DE1-SoC tem, no mesmo chip (Cyclone V SoC), **dois mundos**: um processador ARM de dois núcleos que roda Linux, chamado **HPS** (*Hard Processor System*), e a **FPGA**, onde fica o nosso hardware. Os dois se falam por **pontes** (barramentos AXI) dentro do chip. O computador do aluno fala com o HPS pela rede.

Ninguém no computador toca a FPGA diretamente. O caminho é sempre o mesmo, e vale para a FFT, para o ADC e para qualquer bloco novo:



1. A janela pega o sinal (gerado, de arquivo ou capturado) e chama o protocolo.
2. O protocolo monta um **pedido** binário (cabeçalho de 20 bytes + amostras) e o manda por TCP, na porta 5000, ao servidor da placa.
3. O servidor escreve as amostras nas **memórias** da FPGA, que ele enxerga como se fossem memória comum (`mmap` de `/dev/mem`).
4. O servidor sobe o bit **start** de um PIO (um registrador de 1 bit). O bloco da FPGA vê a borda de subida e começa.
5. O bloco lê as memórias, alimenta o IP da FFT e grava o resultado em outras memórias.
6. O bloco sobe o bit **done**. O servidor, que estava esperando por ele, lê o resultado.
7. O servidor monta a **resposta** e a manda de volta. A janela mostra o resultado e pode salvá-lo para comparar com o NumPy.

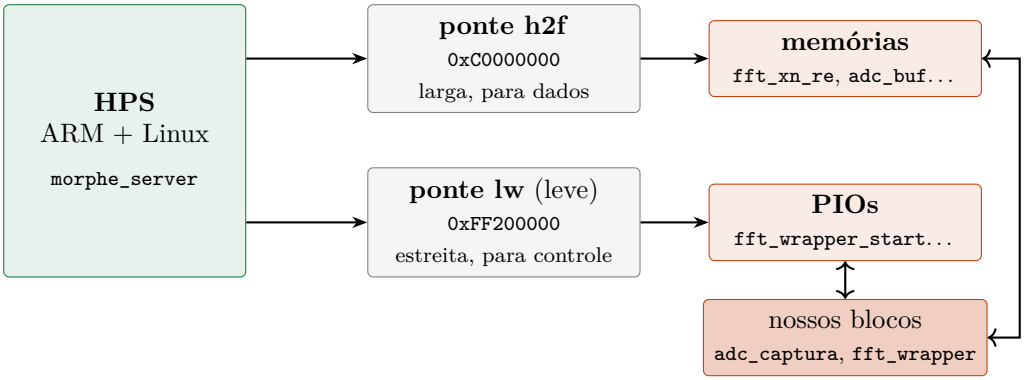
Por quê

Por que um servidor no meio, e não o computador falando direto com a FPGA? A FPGA não tem rede: quem tem placa de rede, Linux e TCP é o HPS. E o HPS alcança a FPGA sem custo, pelas pontes internas do chip. Assim o hardware fica simples (memórias e registradores), o servidor fica pequeno (copiar, disparar, esperar, devolver) e toda a conta *difícil* de interface (gráficos, arquivos, comparação) fica no computador, em Python.

2 A placa por dentro: HPS, FPGA e as pontes

2.1 As duas pontes

O HPS enxerga a FPGA por **duas janelas de endereço**. Cada uma é uma ponte AXI e tem um uso:



- **h2f** (*HPS-to-FPGA*), a partir de `0xC0000000`: aqui ficam as **memórias** (blocos de RAM on-chip). É por onde passam os vetores de amostras.
- **lw** (*lightweight*), a partir de `0xFF200000`: aqui ficam os **PIOs**, registradores de 1 a 32 bits. É por onde passam os comandos (start, configuração) e os avisos (done, contador).

O endereço que aparece no Platform Designer é **relativo** à ponte. O servidor soma a base: a memória `fft_xn_re`, no endereço `0x1a000` da ponte h2f, fica em `0xC001A000` para o ARM. Os dois valores de base estão em `C/address_map_arm.h` (`FPGA_ONCHIP_BASE` e `LW_BRIDGE_BASE`).

2.2 Memória de duas portas e PIO: as duas peças de toda interface

Todo bloco do Morphe conversa com o HPS usando só duas peças do Platform Designer:

Memória on-chip de duas portas (`altera_avalon_onchip_memory2`, `dualPort = true`). A porta **s2** fica ligada à ponte h2f: é a que o HPS lê e escreve. A porta **s1** é *exportada* para fora do sistema e chega ao `ghrd_top.v` como um punhado de fios (endereço, dado, escrita): é a que o nosso bloco usa. Os dois lados acessam a mesma RAM, cada um pela sua porta, sem se atrapalhar.

PIO (`altera_avalon_pio`). Um registrador. Se é de **saída** (*Output*), o HPS escreve e o bloco lê os fios exportados (ex.: `fft_wrapper_start`). Se é de **entrada** (*Input*), o bloco põe o valor nos fios e o HPS lê (ex.: `fft_wrapper_done`).

2.3 O mapa de endereços

Estes são os endereços desta versão, como saem do `C/hps_0.h` (gerado, seção 3.6). Os nomes são os dos componentes no `Quartus/soc_system.qsys`.

ponte	componente	endereço	o que é
h2f	`onchip_memory2_0`	`0x00000`–`0x0FFFF`	RAM do projeto de referência (não usada pelo Morphe)
h2f	`fft_yn_re`	`0x18000` (4 KiB)	saída da FFT, parte real
h2f	`fft_yn_imag`	`0x19000` (4 KiB)	saída da FFT, parte imaginária
h2f	`fft_xn_re`	`0x1A000` (4 KiB)	entrada da FFT, parte real
h2f	`fft_xn_imag`	`0x1B000` (4 KiB)	entrada da FFT, parte imaginária
h2f	`adc_buf`	`0x20000` (128 KiB)	32768 amostras do ADC
lw	`fft_bfp_exponent`	`0x50`	entrada: expoente de bloco da FFT (6 bits)
lw	`fft_wrapper_done`	`0x60`	entrada: FFT terminou
lw	`fft_inverse`	`0x70`	saída: transformada inversa (só diagnóstico)
lw	`fft_wrapper_start`	`0x80`	saída: dispara a FFT
lw	`adc_start`	`0xD0`	saída: dispara a captura
lw	`adc_done`	`0xE0`	entrada: captura terminou
lw	`adc_config`	`0xF0`	saída: palavra do conversor (bits 5..0) e modo contínuo (bit 6)
lw	`adc_divisor`	`0x100`	saída: período de amostragem, em ciclos de 50 MHz
lw	`adc_namostras`	`0x110`	saída: quantas amostras (captura única)
lw	`adc_contador`	`0x120`	entrada: amostras já gravadas na RAM
lw	`sysid_qsys`	`0x10000`	identificador e *timestamp* do sistema

Cuidado

Endereço sem ninguém trava a placa. No Cyclone V, um acesso do HPS a um endereço da ponte onde não existe componente **não dá erro**: trava o barramento inteiro, a placa some da rede e até a serial congela. É por isso que o servidor só toca a FPGA depois de confirmar que ela está com o bitstream do Morphe (seção 2.4), e é por isso que o `hps_0.h` **nunca** é escrito à mão: ele tem que descrever exatamente o hardware que está gravado na FPGA.

2.4 Como o servidor sabe que a FPGA é a certa

Quando a placa liga, a FPGA carrega um bitstream de fábrica que está no cartão SD, e nele os nossos PIOs não existem. Só depois que alguém grava o bitstream do Morphe pelo cabo USB (o aplicativo faz isso ao abrir, ou o `morphe-up.sh`) é que os endereços passam a existir. O servidor tem dois jeitos de saber:

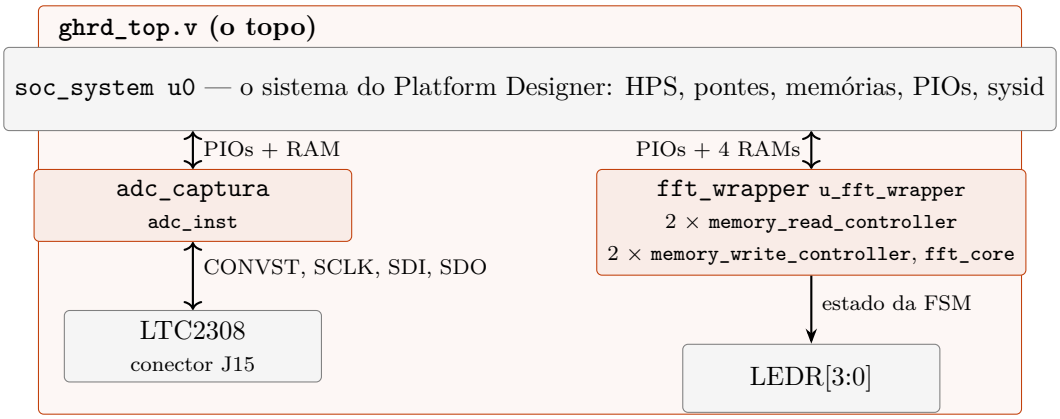
1. **A assinatura** `MRPH`. O `ghrd_top.v` liga a constante `32'h4D525048` ("MRPH" em ASCII) na entrada `h2f_gp_in` do HPS:

   ```
   localparam MORPHE_ASSINATURA = 32'h4D525048;
       .hps_0_h2f_gp_gp_in                     (MORPHE_ASSINATURA),
   ```

   Esse valor aparece num registrador do gerenciador da FPGA (GPI), que existe sempre e pode ser lido sem risco. Se ele vale `MRPH`, a FPGA tem o nosso bitstream.

2. **O timestamp do sysid**. O componente `sysid_qsys` guarda o instante em que o sistema foi gerado. O `hps_0.h` traz o mesmo número (`SYSID_QSYS_TIMESTAMP`). Antes do ADC, o servidor compara os dois: se forem diferentes, o bitstream gravado não é o que este servidor conhece, e a operação é recusada com uma mensagem, em vez de travar a placa.

3 A FPGA (pasta `Quartus/`)



3.1 O topo: `ghrd_top.v`

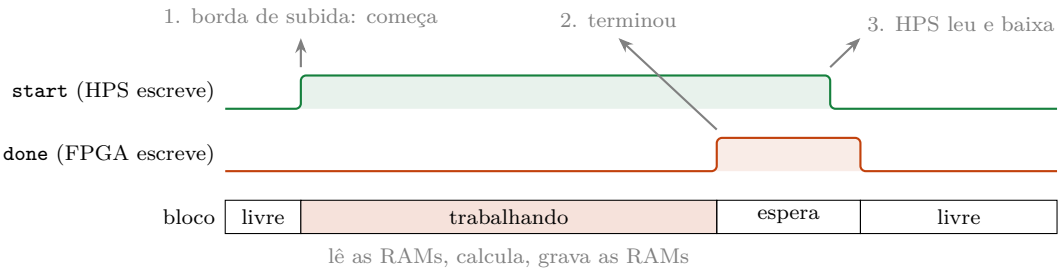
É o arquivo que o Quartus compila como raiz do projeto. Ele faz três coisas, e um módulo novo mexe nas três:

1. **declara os fios** que ligam o sistema do Platform Designer aos nossos blocos (ex.: `wire adc_start;`, `wire [31:0] adc_divisor;`);
2. **instancia os blocos** (`adc_captura adc_inst (...)`, `fft_wrapper u_fft_wrapper (...)`) ligando as portas deles a esses fios e aos pinos da placa;
3. **instancia o sistema** `soc_system u0 (...)` e liga cada porta exportada a um fio. O nome da porta é o nome do componente seguido do nome da interface exportada: o PIO `adc_start` vira `.adc_start_export`, a porta s1 da RAM `adc_buf` vira `.adc_buf_address`, `.adc_buf_writedata`, e assim por diante.

Nesta versão o topo também desliga o que não é usado: os displays `HEX0` a `HEX5` ficam apagados (`7'h7F`), `LEDR[9:4]` em zero, e os fios do I2C do codec de áudio em alta impedância (`1'bz`). `LEDR[3:0]` mostra o estado da máquina da FFT, útil para ver se ela travou.

3.2 O combinado entre o HPS e um bloco: start e done

Os dois blocos usam o **mesmo aperto de mão**, e um bloco novo deve usar também. Ele só precisa de dois PIOs de 1 bit:



1. O HPS escreve os dados nas RAMs e a configuração nos PIOs. Então escreve `start = 0`, espera um instante e escreve `start = 1`. O bloco reage à **borda de subida**, não ao nível: guarda o valor anterior num registrador e compara.

```
reg start_d;
always @(posedge clk or negedge reset_n)
    if (!reset_n) start_d <= 1'b0;
```

```
    else          start_d <= start;
wire start_pulse = start & ~start_d;
```

2. O bloco trabalha e, no fim, sobe `done` e **fica parado** esperando.
3. O servidor lê `done` em laço (`wait_done`, com prazo). Quando vê 1, lê o resultado e baixa `start`. O bloco vê `start = 0`, baixa `done` e volta a ficar livre.

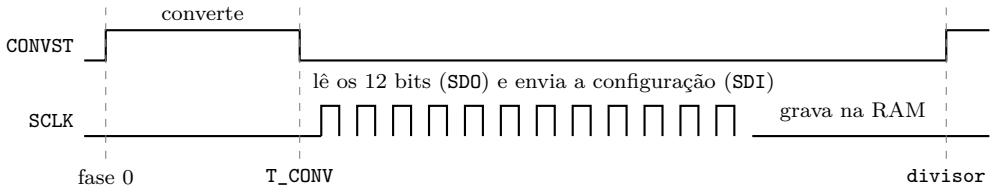
Por quê

Reagir à borda, e não ao nível, impede que o bloco rode duas vezes seguidas se o HPS demorar a baixar o start. Esperar o start baixar antes de baixar o done garante que o HPS *viu* o done: sem isso, um done curto demais poderia passar entre duas leituras do laço e o servidor esperaria até o prazo estourar. É a mesma regra nos dois blocos: `S_COMPLETE` do `fft_wrapper.v` e `S_DONE` do `adc_captura.v`.

3.3 O ADC: `adc_captura.v`

O conversor da placa é um **LTC2308**: 12 bits, 8 canais, no conector J15. Ele se comunica por uma interface serial de 4 fios: `CONVST` (começa a conversão), `SCLK` (relógio), `SDI` (configuração, da FPGA para o conversor) e `SDO` (o resultado, do conversor para a FPGA).

O `adc_captura.v` existe para que cada amostra seja tirada num instante **exato**: um contador de período (`fase`) conta de 0 a `divisor - 1` a 50 MHz, e cada conversão começa no ciclo em que ele zera. Assim $f_s = 50\,\text{MHz}/\text{divisor}$, de 1 kHz (divisor 50000) a 200 kHz (divisor 250).



Captura única (`adc_config[6] = 0`): o HPS escreve `adc_divisor`, `adc_namostras` (até 32768) e `adc_config`, sobe `adc_start`, e o bloco grava as amostras de `adc_buf[0]` em diante. No fim, `done` sobe.

Captura contínua (`adc_config[6] = 1`): a RAM vira um **buffer circular** e a captura não para até o HPS baixar o start. O PIO `adc_contador` diz quantas amostras já foram gravadas; o servidor copia as novas enquanto o hardware segue. Se o servidor atrasar mais que o tamanho da RAM (164 ms a 200 kHz), ele percebe pelo contador e encerra com "perdeu amostras", em vez de entregar um buraco escondido.

Dois detalhes do conversor que aparecem no código:

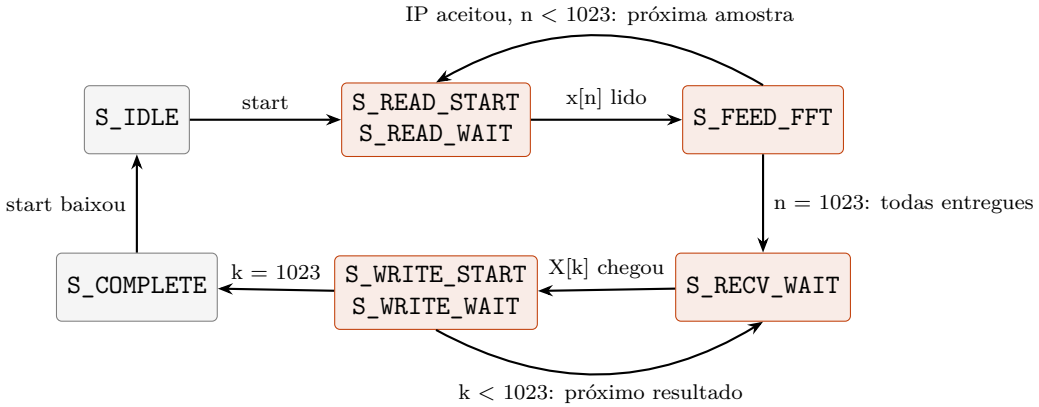
- A palavra de configuração (6 bits: `S/D O/S S1 S0 UNI SLP`: entrada simples ou diferencial, canal, unipolar ou bipolar, *sleep*) enviada num quadro vale para a conversão do quadro *seguinte*. Por isso o **primeiro quadro é descartado**: são $n + 1$ conversões para n amostras.
- A RAM guarda o código cru, nos 12 bits baixos. No modo bipolar ele é complemento de dois de 12 bits, e quem estende o sinal é o servidor (`adc_codigo`).

A pinagem do J15, as faixas de tensão e o que foi medido estão em `docs/ADC.md`.

3.4 A FFT: `fft_wrapper.v` e o IP `fft_core`

O cálculo da FFT é feito por um **IP da Intel** (`fft_core`, configurado em `Quartus/fft_core.qsys`): 1024 pontos, arquitetura *Buffered Burst*, entrada de 24 bits e **ponto flutuante de bloco** (*block*

floating point, BFP). O IP não sabe ler memória: ele recebe e entrega amostras por **Avalon-ST**, um fluxo com sinais de mão (`valid`/`ready`) e marcas de início e fim de quadro (`sop`/`eop`). O `fft_wrapper.v` é a cola entre as memórias e esse fluxo.



- **Alimentar**: para cada n de 0 a 1023, um `memory_read_controller` lê `fft_xn_re[n]` e `fft_xn_imag[n]` (as duas leituras em paralelo), o wrapper põe os 24 bits baixos em `sink_real`/`sink_imag` e espera o IP aceitar (`sink_ready`).
- **Receber**: o IP devolve $X[k]$ e o expoente do bloco; um `memory_write_controller` grava em `fft_yn_re[k]` e `fft_yn_imag[k]`, estendendo o sinal para 32 bits.
- **Terminar**: o expoente vai para o PIO `fft_bfp_exponent` e `done` sobe.

O formato dos números: Q15.8. Cada amostra da FFT é um inteiro de 24 bits com 8 bits de fração: 1 bit de sinal, 15 de parte inteira, 8 de fração. O valor real é o inteiro dividido por $2^8 = 256$, e a faixa vai de -32768 a $+32767{,}996$. Nas memórias de 32 bits, os 8 bits de cima são só a extensão do sinal. O IP, por sua vez, escala o resultado para não estourar e informa quanto escalou pelo expoente e: o valor certo é $y_{\text{bruto}} \cdot 2^{-e}/256$. Essa conta é do servidor (`fft_run_block`). Os números medidos (o teto de ∼92 dB, os 6,02 dB por bit) estão em `docs/PRECISAO-NUMERICA.md`.

Cuidado

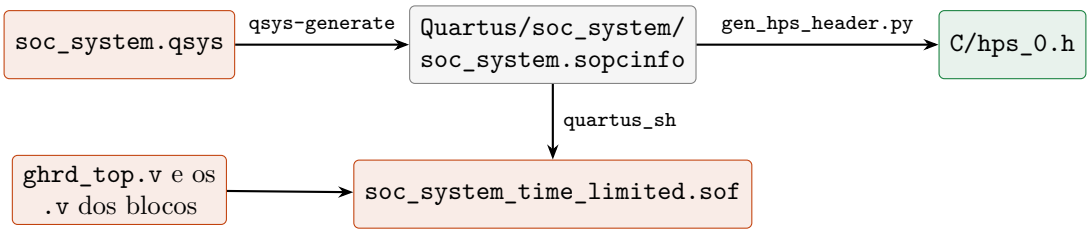
O IP da FFT é licenciado. Sem licença paga, o Quartus compila o IP no modo de avaliação (*OpenCore Plus*): o arquivo gerado é o `soc_system_time_limited.sof`, e a FFT só funciona **enquanto o cabo USB estiver ligado com o programador aberto** (o "tether"). Quem mantém o programador aberto é o aplicativo, ou o `morphe-up.sh`. Desligou o cabo, ou derrubou o tether, a FFT para de responder até gravar de novo. Um bloco seu, escrito só em Verilog, não tem essa limitação.

3.5 O sistema do Platform Designer: `soc_system.qsys`

É aberto no Platform Designer (`Tools` → `Platform Designer` no Quartus 20.1). Nesta versão ele tem: o HPS (`hps_0`), as 6 memórias da tabela acima, os 10 PIOs (4 da FFT, 6 do ADC), o `sysid_qsys`, uma `jtag_uart` e dois mestres JTAG de depuração. Cada memória e cada PIO tem duas ligações: a porta do lado do HPS ligada a uma ponte (memória → h2f, PIO → lw) e a outra *exportada* para o `ghrd_top.v`.

3.6 O que é gerado, e por quem

Três coisas no repositório **não se edita à mão**: elas saem de uma ferramenta, a partir de outra fonte.



`Quartus/soc_system/` O Verilog do sistema, gerado pelo `qsys-generate` (ou pelo botão *Generate HDL* do Platform Designer) a partir do `.qsys`.

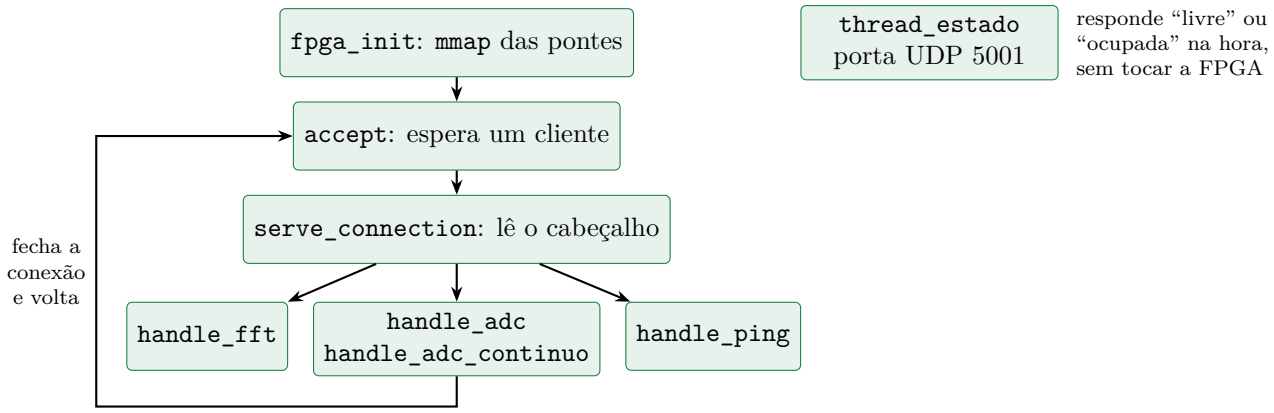
`C/hps_0.h` Os endereços, gerados pelo `Quartus/gen_hps_header.py` a partir do `.sopcinfo`. Se você muda o Platform Designer e não regenera este arquivo, o servidor usa os endereços velhos. `python3 Quartus/gen_hps_header.py --check` confere se os dois estão de acordo.

`.sof` O bitstream, compilado pelo Quartus a partir de tudo acima. Fica em `Quartus/output_files/` e vai para o git, junto com o `PROVENIENCIA.sha256` (as impressões digitais das fontes de que ele saiu). `sha256sum -c PROVENIENCIA.sha256` confere um clone.

A sequência completa, com os comandos, está em `docs/COMPILAR.md`.

4 O servidor (pasta `C/`)

O `morphe_server` é um programa em C que roda no Linux da placa, como `root`, na pasta `/root/morphe`. Ele é compilado **na própria placa** (`make`), com os seis arquivos que o `morphe-up.sh` envia: `morphe_server.c`, `morphe_protocol.h`, `morphe_config.h`, `hps_0.h`, `address_map_arm.h` e `Makefile`.



`fpga_init` Abre `/dev/mem` e mapeia as duas pontes. Depois disso, `g_fft_xn_re[i]` é a posição `i` da memória da FPGA, e `*g_pio_fft_start = 1` escreve no PIO. Os ponteiros são montados a partir do `hps_0.h`:

```
g_pio_fft_start   = (volatile uint32_t *)((char *)g_lw_virt + FFT_WRAPPER_START_BASE);
```

O `volatile` é obrigatório: diz ao compilador que o valor pode mudar sozinho (é o hardware quem escreve) e que cada leitura tem que ir de fato ao endereço.

`serve_connection` Lê o cabeçalho de 20 bytes, confere a assinatura `MRPM` e a versão, e escolhe o tratador pelo **opcode**, num `switch`. Um módulo novo ganha um `case` aqui.

`handle_fft` Recebe 1024 amostras Q15.8, escreve em `fft_xn_re` (e zeros em `fft_xn_imag`), dispara com o aperto de mão da seção 3.2, lê `fft_yn_re`/`fft_yn_imag`, aplica a escala do expoente e devolve 1024 complexos em `float32`.

`handle_adc` Confere a fs, o tamanho e a configuração, escreve os PIOs do ADC, dispara, espera (o prazo sai da duração da captura) e devolve os códigos do conversor.

`handle_ping` Responde um texto `chave=valor` com o nome da placa, os limites do hardware e se a FPGA está preparada. É o que o aplicativo usa para achar e conhecer a placa.

`thread_estado` Uma linha de execução à parte que responde, por UDP na porta 5001, se a placa está livre ou ocupada. Também atende o *broadcast* com que o aplicativo acha todas as placas da rede.

Um cliente por vez. O laço principal atende uma conexão, responde e fecha. Se dois alunos usam a mesma placa, o segundo espera o primeiro terminar; ninguém recebe erro. O aplicativo usa a porta de estado para escolher a placa mais livre.

Arquivos de depuração. Cada FFT grava um `.mrph` em `/root/morphe` com a entrada, a saída bruta e a saída escalada (os 100 mais recentes ficam; a variável `MORPHE_DUMP_MAX` muda o limite).

Os limites ficam num lugar só. Tamanhos do hardware (1024 pontos, 32768 amostras, faixa do divisor, porta) estão em `C/morphe_config.h`, e o espelho deles para o cliente em `Python/morphe_config.py`. Se mudar um, mude o outro.

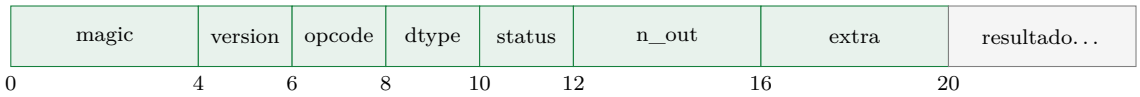
5 O protocolo (`morphe_protocol.h` e `.py`)

Cliente e servidor trocam mensagens binárias pela conexão TCP. Todos os números são **big-endian** (o byte mais significativo primeiro, a "ordem da rede"). Os dois arquivos, o `.h` do servidor e o `.py` do cliente, descrevem o mesmo formato e têm que concordar byte a byte.

pedido (cliente → placa): `magic` = `MRPM`



resposta (placa → cliente): `magic` = `MRPN`



opcode	nome	pedido → resposta
2	`FFT`	1024 amostras `int32` em Q15.8 → 1024 complexos `float32` (re e im intercalados)
3	`PING`	só o cabeçalho → texto `chave=valor` com os dados da placa
7	`ADC`	só o cabeçalho; `n_x` = amostras, `n_h` = divisor, `flags` = configuração → `n_x` códigos `int32`
8	`ADC_CONTINUO`	como o ADC, `n_x = 0` = até o cliente parar → uma sequência de blocos

Os números 1, 4, 5 e 6 eram de blocos da versão completa (convolução, FIR, IFFT, IIR) que esta versão não tem; o 9 já é usado pelo módulo de soma do roteiro (e, na versão completa, pelo áudio). **Um módulo novo usa um número que ninguém usa**, nos dois arquivos ao mesmo tempo.

Se algo dá errado, a resposta vem com `status` diferente de 0 e uma mensagem em texto no lugar do resultado. Os códigos estão no `morphe_protocol.h`; os mais comuns são `BAD_OPCODE` (3: o servidor não conhece aquela operação), `BAD_SIZE` (5), `FPGA_TIMEOUT` (6: o done não subiu no prazo) e `FPGA_NAO_PREPARADA` (8: a FPGA não está com o bitstream do Morphe).

6 O aplicativo (pasta `Python/`)

O aplicativo é aberto com `python3 morphe_app.py`, de dentro de `Python/`. Ele é organizado em camadas, e é bom manter essa separação num módulo novo: **a janela não monta bytes, e o protocolo não desenha nada**.

janelas (Tk) `morphe_app.py` (o menu principal), `signal_generator_window.py`, `aquisicao_window.py`, `fft_window.py`, `comparator_window.py`

contas, sem tela `aquisicao.py` (ADC: fs, volts, métricas), `blocos.py` (FFT longa), `dsp_core.py` (sinais, Q15.8, `.mrph`)

fio `morphe_protocol.py` (monta pedidos, lê respostas, `TcpClient`, descoberta) `morphe_config.py` (limites)

infraestrutura `tcp_panel.py` (painel de conexão, escolha da placa), `discovery_window.py`, `programar_local.py` (grava a FPGA pelo USB)

`morphe_protocol.py` Para cada opcode, uma função `build_..._request` (monta os bytes) e uma `decode_..._response` (devolve um `numpy.ndarray`). O `TcpClient` abre a conexão, manda, lê a resposta inteira e fecha.

A normalização da FFT Antes de converter para Q15.8, `build_fft_request` multiplica o sinal para que o pico encoste perto do teto da faixa, e `decode_fft_response` desfaz a escala na volta. Como a FFT é linear, o resultado é o mesmo; o que muda é que o erro de quantização fica o menor possível.

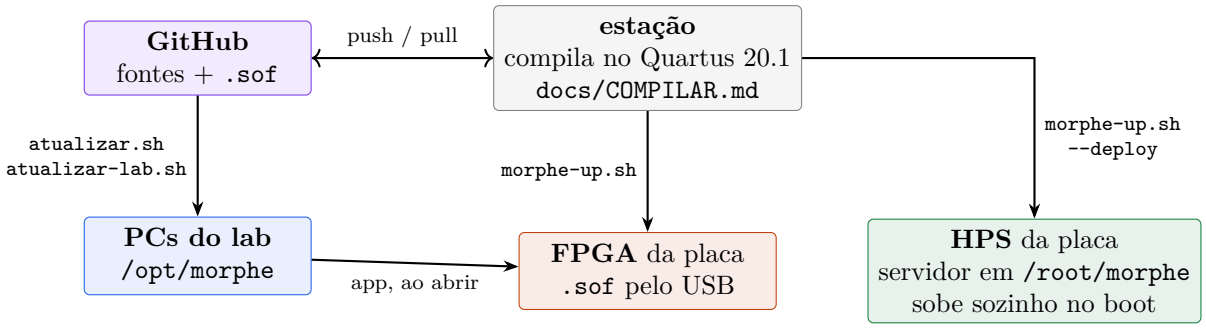
A FFT longa A placa só faz 1024 pontos. `blocos.py` calcula uma FFT de $1024 \cdot M$ pontos pelo algoritmo de quatro passos: M FFTs de 1024 na placa, ligadas no computador por fatores de giro. É exata, não uma aproximação.

`.mrph` O formato de arquivo do Morphe: texto, com seções (entrada, saída...). A janela da FFT salva um; o Comparador o abre e compara a saída da FPGA com o `numpy.fft`.

Achar a placa Ao abrir, o painel manda um *broadcast* UDP (`descobrir_placas`) e cada servidor responde com nome e IP. Por isso o IP da placa pode mudar (DHCP) sem que ninguém precise saber dele.

Gravar a FPGA `programar_local.py` procura, com o `jtagconfig` do Quartus, as placas ligadas pelo USB *deste* computador e grava o `.sof` nelas, mantendo o tether aberto.

7 A infraestrutura: como tudo chega à placa e aos computadores



Compilar Só na estação, com o Quartus 20.1: `qsys-generate`, `gen_hps_header.py`, `quartus_sh`, gravar, testar e só então pôr o `.sof` no git com a proveniência (`docs/COMPILAR.md`).

morphe-up.sh Deixa uma placa pronta em um comando, nesta ordem: acha o Quartus, descobre o cabo (`jtagconfig`), grava a FPGA e mantém o tether, acha a placa na rede, envia e compila o servidor se as fontes mudaram (`--deploy` força), reinicia o servidor e confere com o `morphe_ping`. `--down` encerra os tethers; `--status` mostra o que está no ar. Roda na conta normal, nunca com `sudo`.

Autostart Cada placa tem o servidor no boot (`C/autostart/instala-autostart.sh`). Ele sobe antes de a FPGA estar gravada e espera, respondendo ao ping com `fpga_preparada=0`.

Computadores O aplicativo é instalado em `/opt/morphe` (`instala-morphe.sh`), um clone do git. `atualizar.sh` atualiza um computador; `atualizar-lab.sh`, rodado na estação, atualiza todos por SSH (a lista está em `computadores.conf`).

As placas `placas.conf` lista as placas do laboratório; o nome (*hostname*) é o que identifica cada uma. Placa nova: `docs/NOVA-PLACA.md`.

Cuidado

Servidor e bitstream andam juntos. O servidor é compilado com o `hps_0.h` de um bitstream. Se você grava um bitstream com componentes novos, a placa precisa também do servidor novo (`morphe-up.sh --deploy`), e vice-versa. Servidor de uma versão com bitstream de outra termina em "o bitstream na FPGA não é o deste servidor" ou em `BAD_OPCODE`. Para experimentar, use um **clone seu** (`~/meu-modulo`): o `/opt/morphe` é o dos alunos e não se mexe.

8 Testes que já existem

teste	o que confere
`Quartus/morphe_ping.py`	4 testes contra uma placa: conexão TCP, protocolo (PING), captura do ADC e FFT de um impulso. O `morphe-up.sh` roda no fim ("4/4")
`Python/ferramentas/testa_adc.py`	o ADC com multímetro e gerador de sinais (tensão, senoide, contínua, ENOB)
`Python/ferramentas/testa_blocos.py`	a FFT longa e a leitura de arquivos, contra a placa ou sem ela
`Python/ferramentas/estado_placas.py`	o que cada placa da rede está fazendo agora

Quando algo falha, olhe nesta ordem: a mensagem do aplicativo (ela traz o `status` e o texto do servidor); o log do servidor; os `LEDR[3:0]` (estado da FFT parado num número diferente de 0 = a máquina travou no meio); e o `--check` do `gen_hps_header.py`.

9 Acrescentar um módulo: o que mexer, camada por camada

Um bloco novo atravessa as mesmas camadas que o ADC e a FFT. A ordem abaixo é a que funciona: **cada camada é testada antes da próxima**, e o módulo é testado **sozinho**, pela serial, antes de ser ligado ao aplicativo.

	o que fazer	onde
1	o **módulo em Verilog**	`Quartus/`
2	**memórias e PIOs** no Platform Designer, e *Generate HDL*	`soc_system.qsys`
3	**fios, instância e exports** no topo; o `.v` novo no projeto	`ghrd_top.v`, `soc_system.qsf`
4	**compilar** e regenerar os endereços	`quartus_sh`, `gen_hps_header.py`
5	**testar sozinho pela serial**, lendo e escrevendo os endereços	`devmem` (vem com o roteiro)
6	**opcode e tratador** no servidor	`morphe_protocol.h`, `morphe_server.c`
7	**pedido, resposta e janela** no aplicativo	`morphe_protocol.py`, `*_window.py`
8	**gravar, enviar o servidor e comparar** com o Python	`morphe-up.sh --deploy`

1. **Verilog.** Copie o padrão dos blocos que existem: `clk` e `reset_n`, `start` por borda, `done` até o start baixar, e as portas de RAM iguais às do `adc_captura.v` (`address`, `wdata`, `write`).
2. **Platform Designer.** Para cada vetor, uma memória de duas portas (porta s2 na ponte h2f, s1 exportada). Para cada comando ou aviso, um PIO (de saída para o que o HPS manda, de entrada para o que o bloco devolve), na ponte lw. Escolha endereços livres (abaixo) e gere o HDL.
3. **Topo.** Declare os fios, instancie o bloco e ligue as portas exportadas no `soc_system u0`. O arquivo `.v` novo entra no projeto (`Project` → `Add/Remove Files`, que grava no `soc_system.qsf`).
4. **Compilar** na estação e rodar o `gen_hps_header.py`. Os endereços novos aparecem no `C/hps_0.h`.
5. **Serial.** Com a placa gravada, escreva nos PIOs e leia as memórias pelos endereços absolutos (`0xFF200000` + endereço lw, `0xC0000000` + endereço h2f). Se o número certo aparece, o hardware está pronto; o resto é software.
6. **Servidor.** Um `#define MORPHE_OP_...` novo no `morphe_protocol.h`; os ponteiros no `fpga_init`; uma função `handle_...` que copia, dispara, espera e devolve (a `handle_fft` é o modelo); e o `case` no `serve_connection`.
7. **Aplicativo.** O mesmo opcode no `morphe_protocol.py`, um `build_..._request` e um `decode_..._response`; uma janela que chama os dois e mostra o resultado ao lado do cálculo em NumPy; e um botão no `morphe_app.py`.
8. **Placa.** `morphe-up.sh --deploy` do seu clone grava o `.sof` novo e manda o servidor novo. No fim, `--down` no seu clone e `morphe-up.sh` do `/opt/morphe` devolvem a placa como estava.

Endereços livres

- **Ponte lw (PIOs):** os PIOs desta versão ocupam de `0x50` a `0x12F`. O servidor mapeia a ponte lw até `0x4FFF` (`LW_BRIDGE_SPAN` no `address_map_arm.h`), então um PIO novo vai entre `0x130` e `0x4FF0`, de 16 em 16 (cada PIO ocupa 16 bytes).
- **Ponte h2f (memórias):** estão livres de `0x10000` a `0x17FFF` (32 KiB) e de `0x1C000` a `0x1FFFF` (16 KiB). Uma RAM de 1024 palavras de 32 bits ocupa 4 KiB.

Cuidado

O servidor só mapeia o que ele conhece. O tamanho da janela h2f que o servidor mapeia (`MORPHE_ONCHIP_MMAP_SPAN`, no começo do `morphe_server.c`) é calculado a partir do fim das memórias da FFT e do ADC, e não pode passar de `FPGA_ONCHIP_SPAN` (`0x3FFFF`). Uma memória nos buracos acima já está dentro da janela. Uma memória depois de `0x40000` exige aumentar `FPGA_ONCHIP_SPAN` e incluir a memória nova no cálculo; sem isso, o ponteiro aponta para fora do que foi mapeado.

O que tem que ser igual dos dois lados

aqui	tem que bater com
nome e tipo de cada PIO e memória no `.qsys`	as portas `.nome_export` do `soc_system u0` no `ghrd_top.v`
o `.sof` gravado	o `hps_0.h` com que o servidor foi compilado (sysid)
`MORPHE_OP_...` no `morphe_protocol.h`	`OP_...` no `morphe_protocol.py`
limites no `morphe_config.h`	os mesmos no `morphe_config.py`
o formato dos números no Verilog (ex.: Q15.16)	a conversão no Python e a referência em NumPy

Já está pronto

O **roteiro do módulo de soma** (`docs/roteiro-modulo/`) faz todo este caminho com um bloco pequeno: um somador saturado de dois vetores. Ele traz o código completo de cada camada, os prints do Quartus e do Platform Designer e um teste em cada etapa. Comece por ele antes do seu módulo.

Glossário

Avalon-ST Interface de fluxo da Intel: dados com `valid`, `ready` e marcas de início/fim de quadro. É como o IP da FFT recebe e entrega amostras.

BFP *Block floating point*: um expoente só para o bloco inteiro de 1024 resultados, que diz quanto o IP escalou para não estourar.

bitstream (`.sof`) O arquivo que configura a FPGA. Gravado pelo cabo USB (JTAG); se perde ao desligar a placa.

HPS *Hard Processor System*: o ARM de dois núcleos do Cyclone V SoC, onde roda o Linux e o servidor.

`mmap` Chamada do Linux que faz um trecho de endereços físicos (aqui, as pontes) aparecer como memória comum para o programa.

PIO *Parallel I/O*: um registrador de até 32 bits no Platform Designer, ligado ao HPS de um lado e a fios do nosso bloco do outro.

Platform Designer A ferramenta do Quartus (antigo Qsys) que monta o sistema: HPS, pontes, memórias, PIOs e os endereços de cada um.

Q15.8, Q15.16 Inteiros com fração fixa: 15 bits de parte inteira e 8 (ou 16) de fração. O valor é o inteiro dividido por 2^8 (ou 2^{16}).

tether O programador do Quartus que fica aberto depois de gravar, e sem o qual o IP licenciado da FFT para.